GENERATIVE AI / ASSIGNMENT 2

Complete Master Plan

Clinical criterion matching with a fine-tuned Transformer

12 sequential phases. Kaggle-first research compute. Independent local application. Complete rubric traceability.

Revised: 9 November 2026, Asia/Karachi
Assignment deadline: 10 November 2026; verify Classroom clock time
Local device: RTX 3050 Laptop 6 GB / 16 GB RAM / Windows 11
Research compute: free Kaggle, subject to verified allocation

Planning status
Dataset preparation and a historical synthetic Colab pilot are documented. Kaggle allocation and clinical performance remain unverified. The detailed assignment rubric totals 135 marks despite its 100-mark heading.

Contents

1. Read this first . . . 3

2. Device feasibility and resource policy . . . 4

3. Execution map and phase rules . . . 8

Phase 0 - Establish the environment and project contract . . . 10

Phase 1 - Research and choose the model pair . . . 11

Phase 2 - Prepare the exact dataset and immutable split . . . 12

Phase 3 - Build evaluation and record the unchanged-model baseline . . . 14

Phase 4 - Fine-tune and select a validated checkpoint . . . 15

Phase 5 - Make evidence and explanations traceable . . . 16

Phase 6 - Implement explicit KV-cached greedy generation . . . 17

Phase 7 - Implement tiled attention with online softmax . . . 18

Phase 8 - Implement greedy speculative decoding . . . 19

Phase 9 - Freeze settings, run final evaluation and consolidate evidence . . . 21

Phase 10 - Deliver the standalone browser application . . . 22

Phase 11 - Write the paper, package the submission and rehearse . . . 23

4. Risk controls and scope discipline . . . 24

5. Full-credit deliverables and rubric traceability . . . 25

6. Verified research starting points . . . 27

7. Handoff prompt for completing one phase . . . 28

1. Read this first

Build a small, reproducible clinical-criterion screening system, then prove its correctness and explain its limitations. The strongest submission is a complete system with honest measurements, traceable evidence, and implementations you can defend. A larger model, a speculative speedup, paid compute, and a custom CUDA kernel are not compulsory.

The supplied assignment is the authority for requirements. Its heading says 100 marks, but its detailed rubric totals **135 marks**: A=15, B=20, C=20, D=15, E=15, F=15, G=20, H=15. Cover all 27 five-mark subcriteria. Do not assume a normalization rule; ask the instructor separately without delaying work.

The deadline is **10 November 2026**. The PDF does not state a clock time or deadline timezone. Verify the exact Google Classroom timestamp immediately. A two-hour grace period is stated, but plan to submit before the official deadline. Missing the live demo means no marks under the assignment's rules. Also submit a **5-7 minute recorded demonstration**; the recording does not replace live attendance.

Core decisions

- Use only `ncbi/TrialGPT-Criterion-Annotations` for core supervised training and comparison [S1]. Keep its six expert labels; inclusion and exclusion have different applicable subsets.
- Provisional target: `Qwen/Qwen2.5-1.5B-Instruct`; provisional draft: `Qwen/Qwen2.5-0.5B-Instruct` [S3-S4]. Commit only after a real memory smoke test and tokenizer compatibility check.
- Use **free Kaggle as the primary training and research-compute environment**, subject to a real allocation/fit check. Keep source development, durable artifact copies, and the final browser application on the laptop. Kaggle CLI authentication has succeeded, but GPU allocation has not yet been verified; this plan does not promise one.
- Prefer ordinary 16-bit LoRA on the allocated GPU first. If it cannot fit, use verified 4-bit LoRA or the documented local fallback. Keep inference comparisons at the same target checkpoint and precision; extra cloud memory is not a reason to select a model the final laptop cannot serve.
- Use PyTorch, Transformers, PEFT, NumPy, scikit-learn and a local Gradio browser application. These are proposed implementation choices; pin versions only after compatibility checks. A locally hosted browser app meets the assignment.
- Implement the cache loop, tiled online-softmax attention, and greedy speculative algorithm yourself. Libraries may supply model layers, cache containers and reference tests.
- Keep educational attention experiments separate from the selected model unless integration is necessary. Do not claim the model uses the custom attention if it does not.
- Generate label and sentence IDs with the model. Recover quoted evidence by looking up original sentences. Use restrained, evidence-linked explanation templates because this dataset has no expert-written explanation target.
- Perform development on train/validation and synthetic fixtures. Run final test evaluation only after configuration freeze.

What the final system does

Patient text + typed criteria -> numbered patient sentences -> fixed prompt -> trained decoder -> label and sentence-ID JSON -> schema/evidence checks -> exact evidence excerpts + explanation -> explicit preliminary aggregation -> browser display and downloadable report.

The same inference service exposes uncached greedy, cached greedy, and speculative greedy generation. An independent attention module demonstrates dense versus tiled attention and optionally compares a supported PyTorch backend. Benchmark charts always come from saved measurements or a live run.

Compute decision: training in Kaggle is compatible with the assignment's allowance for notebooks and free environments. The final evaluator must launch the independent local app; they must not need to connect, authorize cloud storage, execute a notebook or acquire a Kaggle GPU. The detailed comparison and recovery protocol are included below and in `compute_strategy.md`.

2. Device feasibility and resource policy

Observed on this machine

These are read-only observations from 8 November 2026, not benchmark results. Available memory, storage and power conditions can change.

Component	Observation	Implementation consequence
OS	Windows 11 Home, version 10.0.26300	Prefer native Windows; avoid introducing Docker/WSL unless an actual dependency requires it.
CPU	Intel Core i5-13420H, 8 cores / 12 logical processors	Suitable for data preparation, app hosting and small CPU attention tests.
System RAM	15.65 GiB total; 2.31 GiB free at inspection	Prefer cloud training; close memory-heavy apps for local model loading and demo.
GPU	RTX 3050 Laptop, 6,144 MiB VRAM	Small target and draft; batch-one inference; prove fit empirically.
GPU snapshot	439 MiB used; driver 610.74; 30 W reported power cap	Latency estimates from other GPUs are unreliable. Record power/temperature during benchmarks.
Storage	C: 7.48 GiB free; D: 96.51 GiB free	Place environment, model caches, temporary downloads and outputs on D:.
Python	3.14 default; 3.13 also installed	Use a dedicated Python 3.11 environment for broad dependency compatibility. Do not modify system Python.
Tools	uv, node, PDF libraries present; no torch in default interpreter	Provision and validate a dedicated ML environment. No training has been run.

Use D: for `HF_HOME`, `UV_CACHE_DIR`, `PIP_CACHE_DIR`, and task-specific temporary files. Aim to retain at least 20 GiB free on D: and avoid filling C:. Reserve roughly 15-25 GiB for environment, models, adapters, logs and downloads; this is a planning allowance, not an observed installation size. Do not delete unrelated files automatically.

Use AC power and stable cooling. Close unnecessary applications with the user's normal controls. Begin with `num_workers=0`, batch size 1, one loaded target, and one serialized inference request. Training and app serving should run separately.

Kaggle opportunity and allocation decision

Recommendation: switch the primary compute route to free Kaggle, while retaining the local app. With 6 GiB VRAM and little currently free system RAM, local training has tight activation/workspace margins. A larger allocated GPU can reduce that pressure and allow a cleaner 16-bit LoRA experiment. This is a feasibility advantage, not a measured speedup. Kaggle CPU RAM is separate from VRAM; inspect both rather than assuming all resources improve.

Kaggle provides managed notebook jobs and CLI submission; current GPU quota, internet access, session limits and assigned hardware must be checked on the account. Do not assume a fixed weekly allowance, device, or continuous availability. See Kaggle setup.

Option	Opportunity	Cost / failure mode	Decision
Laptop only	Stable ownership and direct final-app testing	6 GiB VRAM, low free RAM, laptop power/thermal constraints	Keep for app, local correctness checks and fallback training.

Option	Opportunity	Cost / failure mode	Decision
Free Kaggle only	Managed Linux compute; more headroom if a suitable GPU is allocated	Allocation uncertainty, restarts, artifact recovery and unsuitable final hosting dependency	Use for research compute, not the complete delivery architecture.
Kaggle compute + laptop app	Training headroom plus independent final delivery	Two runtime profiles and a tested artifact transfer	Preferred route.

The assigned GPU matters more than the Kaggle name. In phase 0 record GPU model, free/total VRAM, compute capability, CPU RAM, disk and current package versions. Then follow this project policy, which is not a Kaggle resource guarantee:

- **At least 12 GiB usable VRAM:** first try 16-bit 1.5B LoRA, batch 1, accumulation 8, gradient checkpointing and 1,024 total tokens. Test 2,048 only if data lengths warrant it and a pilot fits. Preserve the chosen effective batch when changing microbatch.
- **6 to under 12 GiB usable VRAM:** begin at the same short envelope, measure a 10-step pilot and use the fallback ladder if needed. Do not assume an assigned accelerator outperforms the laptop.
- **No GPU, or an unusable runtime:** time-box setup/allocation troubleshooting to 20-30 minutes, then continue CPU development and use the local training fallback. Revisit available free compute at a natural break; do not spend the deadline repeatedly requesting sessions.

Use FP16 when native BF16 is unavailable. Check capability and finite operations on the actual GPU; do not inherit a model card's BF16 dtype blindly. The model pair remains the proposed 1.5B target and 0.5B draft because the final app must fit the 6 GiB laptop. A historical Colab T4 synthetic pilot exists; Kaggle allocation and fit remain unverified.

Two runtime profiles and durable handoff

Keep one portable Python source tree and one set of dataset/model/prompt definitions. Use `configs/runtime_kaggle.yaml` for research jobs and `configs/runtime_local.yaml` for the app. Record shared library versions plus separate tested Linux/Windows environment locks. Prefer Kaggle's existing compatible PyTorch; do not reinstall its entire stack reflexively. Capture the detected Python version and restart/retest after any required package change. Use the dedicated local Python 3.11 environment for Windows compatibility.

```
Local D: workspace -> pinned Git commit / source archive -> Kaggle /kaggle/working project
Kaggle scripts -> checkpoints + predictions + raw logs -> verified persistent bundle
Persistent bundle -> local artifacts/ and results/ -> offline browser application
```

`scripts/prepare_kaggle_job.py` is a private Phase 2 job builder; future job launchers must be implemented and verified. It embeds a checksummed source archive, installs dependencies and runs the doctor and Phase 2 verification. Training, durable checkpointing and resume remain future work. Core data/training/decoder/attention code belongs in Python modules, not notebook cells. A source archive is acceptable until the repository exists; record its hash. Authentication is local; the actual Kaggle run remains to be verified.

Use `/kaggle/working` for active caches, model files, data and checkpoints. Save finished job outputs, download them through the CLI and verify checksums. During-run persistence and resume must be implemented and tested before long training; finished notebook output alone is insufficient for interrupted jobs. Copy archives rather than training against many small remote files. Do not treat autosaved notebook output as saved model weights. Check destination capacity before a run.

Implement a durable checkpoint every **25 optimizer steps or 5 minutes, whichever comes first**, and at each epoch end. Save the adapter, optimizer/scheduler/scaler state, all RNG states, epoch/global step and sampler/order/cursor state needed for resume. Save into a local temporary bundle, copy to a uniquely named persistent destination, verify checksums there, then write a completion marker and update the latest pointer. Keep the last two verified resumable bundles and the best adapter; prune only after a newer copy is verified. Do not repeatedly archive base-model weights that can be downloaded by pinned revision.

Before a long run, simulate interruption and restore a checkpoint. Verify step progression, optimizer state, data position and config hashes. Exact bitwise reproduction across a different GPU is not promised; record any device change. After

training/evaluation, copy the selected adapter, manifests, split manifest, prediction files, logs and figures to the laptop and verify SHA-256 hashes. Load that exported adapter locally before considering cloud training complete.

Resume optimizer state only with a compatible training representation and configuration. Moving from 16-bit LoRA to a 4-bit base is a changed experiment, not an exact resume: document a new run/stage, verify adapter compatibility, reset incompatible optimizer/scaler state and preserve the original lineage. Never silently load incompatible state to claim continuous training.

Fair comparisons across cloud and laptop

Run the primary research benchmark suite on one recorded environment per experiment and compare modes on that same device/session. Never divide a Kaggle speculative time by a laptop greedy time. If a runtime reconnect changes GPU/backend, create a new benchmark group and rerun all compared modes for that group; exclude checkpoint/artifact transfers from model timers and report transfer/setup separately.

Use validation/synthetic fixtures to compare the exported model on both environments before test freeze. Match model, adapter, tokenizer, prompt, dtype and decoding settings where supported; investigate differences. Exact cached/uncached/speculative equality is required within each supported runtime, while cross-device floating-point discrepancies need an explicit explanation. Keep local-app measurements labeled RTX 3050 and cloud measurements labeled with the actual allocation. If local serving needs a different quantization, treat it as a separately identified deployment variant and validate it on development data; do not silently present cloud quality scores as its measured quality.

Model comparison and memory reasoning

Candidate	Architecture and size	Context / license	Role and feasibility
Qwen2.5-1.5B-Instruct	1.54B parameters; 28 layers; hidden 1,536; 12 Q heads / 2 KV heads; head dim 128	32,768 context; Apache-2.0	Preferred target. Raw 16-bit weights about 2.87 GiB; better capacity hypothesis, to test on validation.
Qwen2.5-0.5B-Instruct	0.49B parameters; 24 layers; hidden 896; 14 Q heads / 2 KV heads; head dim 64	32,768 context; Apache-2.0	Smaller target candidate and proposed draft. Raw 16-bit weights about 0.91 GiB.

Both are autoregressive decoder Transformers using grouped-query attention, RoPE positions, RMSNorm and SwiGLU, with tied embeddings. The family uses a Qwen BPE tokenizer; verify exact token-to-ID maps and special tokens rather than assuming compatibility from model names. Model-card values and configs support the specifications above [S3-S4]. Use the model-specific 32K context, not a larger family-wide marketing maximum.

Weight estimate = parameters x bytes per parameter. The target/draft pair consumes about **3.78 GiB in raw 16-bit weights alone**. Runtime also needs caches, logits, activations, temporary kernels, adapter weights and CUDA allocations. Full Adam fine-tuning with weights, gradients and optimizer state at roughly 12-16 bytes per parameter would already require about 17-23 GiB for the target before activations. Ordinary full fine-tuning is unsuitable here. LoRA freezes the base model and updates small low-rank matrices [S6].

For batch size B, cached length T, layers L, KV heads Hkv, head dimension d and element bytes s:

```
KV bytes = 2 * B * T * L * Hkv * d * s
```

At B=1, T=2,048, s=2, the target cache is approximately 56 MiB; draft approximately 24 MiB. These exclude workspace and allocator overhead. Large vocabulary logits, especially full-prompt training logits, can be a substantial extra allocation. Cache savings do not make long-sequence training free.

Initial operating envelope: on Kaggle, begin with 1,024 total training tokens, batch 1, accumulation 8, 16-bit base weights, FP32 adapter/softmax accumulation where appropriate, gradient checkpointing, no training cache. Test 2,048 tokens only after measuring. Local fallback training starts at the same short envelope and may require 4-bit LoRA. Inference starts at prompt cap 1,024 and output cap 96; increase if validation shows truncation or incomplete JSON. Training length includes prompt plus answer. Validate local app limits independently; a cloud-only long-context benchmark is not a promise that the app supports it.

Fallback ladder: use the verified free Kaggle allocation -> reduce unnecessary padding/workspace and verify mixed precision/checkpointing -> use verified 4-bit LoRA if needed [S7] -> resume the same project locally at a measured safe envelope if Kaggle is unavailable. Audit evidence loss before reducing length. Kaggle availability is unverified; use the local fallback if allocation remains unavailable. If the 1.5B target remains infeasible, select a smaller target and an actually smaller tokenizer-compatible draft before freezing baseline/training; repeat affected experiments. Never use the same-sized model as a nominal smaller draft or silently swap targets mid-comparison. Local inference feasibility is a selection gate even when cloud training succeeds.

The CUDA version printed by `nvidia-smi` is driver capability, not proof of an installed CUDA toolkit. Install a supported PyTorch CUDA wheel using official instructions [S11], then verify `torch.cuda.is_available()` and an actual GPU operation. No third-party FlashAttention package is required.

3. Execution map and phase rules

Work in order. Every phase below contains its own contract, implementation steps, outputs and acceptance gate. A phase may consume completed artifacts but must not require a future phase's code. Commands shown are **interfaces to implement**, not claims that those scripts already exist. Use the project environment when invoking them.

Phase	Outcome	Suggested focused effort
0	Reproducible environment and project scaffold	1-2 h
1	Research ledger and feasible model decision	1-2 h
2	Clean data, exact split and stable schemas	1.5-2.5 h
3	Baseline plus trusted evaluation harness	1.5-2.5 h
4	Validated fine-tuned adapter	2-4 h plus measured training
5	Verified evidence and explanation pipeline	1-2 h
6	Correct explicit KV-cache decoder	2-3 h
7	Correct tiled online-softmax attention	2-3 h
8	Correct speculative decoder	3-5 h
9	Frozen experiments and final held-out evaluation	1.5-3 h plus inference
10	Standalone browser app and export	2-4 h
11	IEEE paper, release, video and defense	4-6 h

Total original implementation estimate: roughly 23-42 focused hours plus training/evaluation. Add an initial 30-60 minute Kaggle bootstrap/transfer/resume check; actual compute savings are unknown until the pilot. As of 9 November, completing all work before 10 November is high risk if implementation has not begun. Kaggle may improve compute feasibility but does not eliminate coding, correctness, report or demo work. Record paper notes and figures at each completed phase.

Revised calendar from 9 November: first confirm the Classroom time and finish phases 0-2, including Kaggle access and local inference fit. Use the first productive GPU session for phases 3-4 and export the adapter immediately. Finish phases 5-8 next with CPU tests before allocating GPU for real checks. On 10 November reserve time for frozen evaluation, local app completion, paper/video and at least three hours of submission buffer. If unfinished coding remains, use CPU development between checkpointed research jobs. This preserves phase acceptance boundaries; do not run a dependent phase on an unaccepted artifact. Cut optional sweeps, deployment and visual polish immediately if the clock does not fit.

At the end of each phase, save `handoffs/phase_XX.md`: completed commands, artifact paths/hashes, exact config and dependency versions, acceptance evidence, observed failures, and any unresolved issue. A failed acceptance gate means the phase is incomplete. Update `research/decision_log.md` with the alternative, evidence and decision. Keep these within the same individual project; the student must understand all AI-assisted work.

Stable project layout

```
configs/                 data, model, train, generation, benchmark configs
scripts/                 Kaggle job builder and portable project scripts
src/clinical_match/      data, prompts, schemas, metrics, training, screening
src/clinical_match/decoding/   uncached.py, cached.py, speculative.py, cache_ops.py
src/clinical_match/attention/  dense.py, tiled.py
scripts/                 executable data/train/eval/benchmark/download tools
tests/                   meaningful correctness and integration fixtures
data/raw/                immutable download; usually gitignored
data/processed/          development JSONL and protected test artifact
data/splits/             patient manifest and count summaries
artifacts/               adapters and manifests; large weights outside Git
results/                 baseline, training, evaluation and benchmark outputs
app/                     browser UI and launcher support
report/                  IEEE LaTeX, bibliography, figures, compiled paper
research/                source summaries, decision and failure logs
handoffs/                one completion record per phase
```

Phase 0 - Establish the environment and project contract

Goal: establish reproducible Kaggle research and local application runtimes. **Inputs:** assignment PDF, this plan, device inventory, the user's free Kaggle account if available. **Rubric:** A2, H2 groundwork. **Execution:** local scaffold plus Kaggle job bootstrap; no training yet.

1. Confirm the exact Classroom deadline and live-demo arrangements. Save the interpretation of the 135-mark rubric in `research/requirements.md`.
2. Create the project directories and Git ignore rules for environments, caches, raw data, weights, private inputs and large temporary files. Initialize Git locally if needed. Do not publish credentials or patient uploads.
3. Locally create a Python 3.11 `.venv` on D:, with D: caches and a supported CUDA wheel for app inference [S11]. In Kaggle select a GPU runtime, inspect its actual allocation/Python/PyTorch, and keep a compatible preinstalled torch. Time-box unavailable-GPU troubleshooting to 20-30 minutes; retain the local route.
4. Create portable `scripts/doctor.py` to record OS, Python/packages, CPU RAM, GPU/VRAM, compute capability, CUDA, disk and dtype support. Run GPU matrix multiplication and one tiny forward pass on each available GPU environment. Save separate `results/environments/local.json` and `kaggle_<run_id>.json`; keep `results/environment.json` as a pointer/summary to the primary research environment.
5. Pin compatible shared libraries and tested environment-specific locks. Create `configs/runtime_kaggle.yaml` and `runtime_local.yaml`; make all source paths configurable. Inspect the installed cache API. Do not copy Windows CUDA-wheel assumptions into Linux or force Kaggle onto local Python 3.11.
6. Create `scripts/prepare_kaggle_job.py` as the Phase 2 verification job builder; implement future train/eval/benchmark jobs and tested durable snapshot/resume separately. Code runs in `/kaggle/working`; verify completed job outputs after downloading them. Interrupted-run persistence is not yet proven. Implement checkpoint/export helpers and a synthetic save/restore test now; use 25-step/5-minute persistence later. Keep secrets out of source and notebook outputs.
7. Design the local Windows `.cmd` setup/app launcher. The evaluator will double-click it without needing Kaggle. Record a source archive hash if GitHub publication is not ready.

Outputs: shared dependencies and two environment locks/profiles, `.gitignore`, source skeleton, thin Kaggle launcher, environment reports, checkpoint/export helpers, setup notes and `handoffs/phase_00.md`.

Verify: run `python scripts/doctor.py --output results/environments/<environment>.json` in each runtime. Local caches use D:, Kaggle active files use `/kaggle/working`; verify job outputs by download. A Kaggle interruption/resume test remains outstanding. Record any unavailable cloud GPU explicitly. This proves setup, not training fit.

Done when: local setup reproduces, Kaggle bootstrap, output retrieval and persistence are tested if available, and the active primary/fallback route is explicit. A denied cloud allocation does not block acceptance if the tested local route is ready. No future module is required.

Phase 1 - Research and choose the model pair

Goal: make a defensible architecture choice with measured feasibility. **Inputs:** working environment and hardware report. **Rubric:** A1-A3, F1.

Execution: train-fit pilot on allocated Kaggle GPU; inference-fit pilot on the laptop. Use only synthetic/development fixtures. The model must satisfy both roles before selection.

1. Read and summarize at least five credible sources. Start with TrialGPT for task framing, model cards/configs for architecture, LoRA for adaptation, the caching documentation, online-softmax/FlashAttention papers and speculative decoding [S2-S10]. For each record: claim, relevance, limitation, implementation consequence and citation. Summarize in your own words.
2. Compare the two proposed open-weight decoder candidates using the model table in section 2. Include exact revision, architecture, parameter count, tokenizer, context, license and expected memory. Explain why BERT-style encoders do not directly satisfy the required autoregressive decoding work.
3. Download pinned configs/tokenizers to each runtime's cache (`/kaggle/working` in Kaggle, D: locally). Compare complete vocabularies, added tokens, special IDs and representative encodings. Use identical target-formatted prompt IDs after compatibility is proven.
4. On the primary research GPU run short greedy inference at 256/1,024 prompt tokens and a 10-step LoRA fit test. Start with 16-bit weights, batch 1, accumulation 8, 1,024 total tokens and checkpointing. Record actual VRAM/time/finite values; enable BF16 only when supported. Estimate run cost from this pilot rather than a guessed Kaggle GPU.
5. On the laptop test target inference and target+draft co-residency at the intended app envelope. Training needs only the target; speculative inference needs both. Record cloud and local fit separately. A cloud-fit/local-OOM pair is not accepted for final delivery without an explicitly validated deployment variant or a revised model choice.
6. Freeze `configs/model.yaml`, the provisional prompt schema, and the source ledger. Record rejected choices and observed issues, including any free-cloud constraints.

Outputs: `research/sources.md`, `research/model_comparison.md`, `research/decision_log.md`, `configs/model.yaml`, tokenizer compatibility evidence and `results/feasibility.json`.

Verify: the selected target supports forward logits, explicit KV-cache inputs/outputs and LoRA; the draft is strictly smaller and token-compatible; the chosen runtime fits the measured envelope. A model pair is a hypothesis until these checks pass.

Done when: model choice, precision and compute route have evidence, and no paid service is necessary. Read no held-out patient cases during research or smoke tests.

Phase 2 - Prepare the exact dataset and immutable split

Goal: establish trustworthy supervised examples and contracts. **Inputs:** prescribed dataset ID [S1], selected tokenizer and model config. **Rubric:** B1-B2.

Execution: CPU locally or in Kaggle. Do not occupy a scarce GPU session solely for data cleaning. Generate one canonical split manifest, hash it, and copy it unchanged to every runtime; do not create environment-specific splits.

The inspected dataset metadata reports **1,015 rows and 53 patients**. Using the required patient split gives **603 / 206 / 206 rows** in train / validation / test in the current snapshot. Planning inspection derived labels/formats from train+validation only; test notes, labels and evidence were not inspected. Recompute and publish counts against the pinned revision.

1. Implement `scripts/prepare_data.py`. Download a pinned dataset revision; save source hashes, license, original annotation IDs and raw data checksum. The Hugging Face split named `train` is the entire source, not the required assignment train subset. Ignore its legacy `training` boolean for splitting.
2. Construct the split exactly:

```
patient_ids = sorted(set(all_patient_ids))
assert len(patient_ids) == 53
np.random.default_rng(42).shuffle(patient_ids)
train_ids = patient_ids[:37]
validation_ids = patient_ids[37:45]
test_ids = patient_ids[45:]
assert len(test_ids) == 8
```

3. Save patient IDs, annotation IDs and row counts in `data/splits/manifest.json`. Assert disjoint patient membership, full row coverage, unique annotation IDs, stable reproducibility and a single consistent note per patient or a documented source issue. Trial IDs may recur across splits because the mandated grouping is by patient; disclose this limitation rather than changing the split.
4. Preserve original numbered patient sentences. The development notes use lines such as `0. ...`, `1. ...`. Parse anchored line prefixes; do not naively split every full stop or renumber sentences. Validate every expert evidence ID against that patient's sentence map. Fail loudly on malformed data; never drop troublesome rows silently.
5. Parse `expert_sentences` strings such as `[0, 1, 2]` with safe literal parsing or a strict list parser, never `eval`. Store a sorted unique integer set and retain original source value for audit.
6. Allowlist model inputs: patient note/sentence map, criterion type, criterion text. Retain annotation/patient/trial IDs only for provenance. Expert label and evidence IDs are targets, never prompt inputs. Exclude `gpt4_explanation`, `gpt4_sentences`, `gpt4_eligibility`, `explanation_correctness`, legacy `training`, and all equivalent derived leakage fields. Do not use GPT outputs as explanation targets.
7. Keep the exact categories. Inclusion: `included`, `not included`, `not enough information`, `not applicable`. Exclusion: `excluded`, `not excluded`, `not enough information`, `not applicable`. Do not invert exclusion semantics. Validate type-specific output membership.
8. Measure token lengths on training/validation using the exact chat template. Prefer full notes; if over length, use a fixed sentence-preserving truncation rule decided without test labels. Preserve criterion/instructions and answer space. Report removed sentence counts and training/validation gold-evidence retention; do not choose sentences using gold evidence. Later report the frozen policy's test retention only during final evaluation. Missing evidence remains a possible unknown, never automatic exclusion.

Contracts to create now

```
{
  "annotation_id": "original-id",
  "patient_id": "original-id",
  "trial_id": "original-id",
  "criterion_type": "inclusion",
  "criterion_text": "Age 25 to 65 years",
  "sentences": [{"id": 0, "text": "A 58-year-old patient ..."}],
  "target": {"label": "included", "evidence_sentence_ids": [0]}
}
```

The example is schematic; preserve source ID types in real files. The target Transformer generates only compact JSON such as `{"label":"included","evidence_sentence_ids":[0]}`. Use deterministic field order and sorted evidence IDs. Prompt instructions list allowed categories for the criterion type, request no invented facts and identify the patient text as data.

Define a separate inference result with `raw_output`, parsed label/IDs, parse status, evidence-validation status, exact excerpts, explanation, truncation flag, model/prompt revisions and timing. Gold targets must not enter that inference object until evaluation joins by annotation ID.

Outputs: immutable raw snapshot, split manifest, train/validation JSONL, protected test JSONL, `schemas/`, dataset audit and preprocessing tests. Restrict development commands to train/validation by default; final evaluation requires an explicit flag and frozen-config hash.

Verify: `python scripts/prepare_data.py --config configs/data.yaml`; `python -m pytest tests/test_data.py`. Repeat split creation and compare its hash. Inspect development fixtures for decimal numbers, multiline text, missing/empty evidence and excluded leakage columns.

Done when: every row is accounted for, no patient leaks, sentence IDs round-trip, schemas are stable and test content is sealed from development.

Phase 3 - Build evaluation and record the unchanged-model baseline

Goal: know how quality is measured before training. **Inputs:** processed train/validation, schemas, selected unadapted target. **Rubric:** B4, C1-C3 groundwork.

Execution: CPU metric tests locally; baseline generation on the primary research GPU. Save predictions/configs to persistent storage and the laptop. To conserve research compute, full quality evaluation may use a library's cached deterministic greedy path after checking it against the explicit uncached reference on small fixtures. Record that path and apply identical settings to base/adapted comparisons; it does not satisfy the later requirement to implement cache-aware decoding yourself.

1. Implement prompt construction and an ordinary deterministic uncached greedy reference using model forward passes: at each step feed the complete committed prefix with no reused cache, take last-position argmax, append one token, stop at EOS or the cap. Set `model.eval()` and inference mode. Begin batch-one with no padding. Record exact prompt token IDs.
2. Implement strict JSON parsing and type checks. Keep malformed, truncated, illegal-category and invalid-ID outputs in the results. Do not discard failures or convert them into `not enough information`. Preserve raw output. For scoring, an invalid category is an incorrect prediction; represent it as `__invalid__` in an extra confusion-matrix prediction column.
3. Implement classification accuracy, macro-F1 and per-class precision/recall/F1. Overall macro-F1 uses the six expert classes; inclusion/exclusion analyses use their four applicable classes. Always include those fixed classes, disclose zero-support classes, use `zero_division=0`, and retain all rows. Do not inflate macro-F1 by omitting difficult labels.
4. Implement evidence precision/recall/F1 on annotation-scoped ID sets; the same sentence ID in different records is not the same retrieval event. Let TP=sum intersection sizes, FP=sum predicted-minus-gold sizes, FN=sum gold-minus-predicted sizes. Micro metrics use these totals; macro metrics average per-record scores. Set both-empty per-record P/R/F1=1; if only one set is empty, set all three to 0. For globally empty totals, apply the same explicit convention. Also report nonempty-gold subset results and counts so empty sets cannot hide weak retrieval.
5. Score invalid evidence IDs as false positives, not silent removals. An unparseable evidence list yields an empty predicted set for retrieval scoring and a separate parse-failure count. For UI display, only resolvable IDs produce excerpts. Report raw and validated validity rates separately.
6. Hand-check a tiny metric fixture: correct/incorrect labels, invalid label, exact evidence, partial overlap, empty/empty, empty/nonempty and out-of-range IDs. Establish expected TP/FP/FN and macro values by hand.
7. Run the unchanged selected target on the entire validation split with frozen prompt version v1 and greedy settings. Save outputs, metrics, timing and failure rates. Optional prompt comparison uses validation only, equal token budgets, and a written decision; save a fresh final baseline if the prompt changes.

Outputs: `src/clinical_match/metrics.py`, reference decoder, prompt/parser modules, hand-verified metric tests, `results/baseline/validation_predictions.jsonl`, metrics and config hashes.

Verify: `python -m pytest tests/test_metrics.py tests/test_schema.py`; `python scripts/evaluate.py --split validation --model base --config configs/eval.yaml`. Every validation annotation has exactly one output or an explicit runtime failure, and metrics reproduce from saved predictions without rerunning the model.

Done when: a future adapter can be compared using exactly the same harness. Do not run test evaluation now.

Phase 4 - Fine-tune and select a validated checkpoint

Goal: learn expert labels and sentence IDs using the mandated training patients. **Inputs:** final baseline prompt, processed train/validation, selected target and working environment. **Rubric:** B3-B4, A3.

Execution: free Kaggle GPU is primary, tested laptop fallback. Use the accepted runtime profile, immutable source/split/config hashes, thin notebook launcher and durable checkpoint helper. Do not reselect a model merely because cloud memory is available.

1. Implement executable `scripts/train.py`; the Kaggle notebook invokes this script instead of duplicating training logic. Render the exact chat template with an assistant JSON target. Mask prompt and padding labels with `-100`; apply causal token cross-entropy to assistant answer tokens, including the correct end-of-turn token. Verify the shift/mask with a decoded sample. No GPT explanation supervision is permitted.
2. Starting configuration: LoRA rank 8, alpha 16, dropout 0.05, `q_proj` and `v_proj`, learning rate 2e-4, AdamW, weight decay 0.01, warmup ratio 0.05, max gradient norm 1, microbatch 1, accumulation 8, maximum 3 epochs, seed 42. Use FP16 with a gradient scaler or BF16 only after capability/finite-operation checks. Set training `use_cache=False` and enable gradient checkpointing. These are starting hypotheses, not tuned results.
3. Confirm only intended adapter parameters train. Log trainable/total counts, tokens per batch, loss, learning rate, gradient norms, epoch/step time and memory. Persist a verified resumable bundle every 25 optimizer steps or 5 minutes, whichever is first, plus epoch ends. Include adapter, optimizer/scheduler/scaler, RNG and data-order/cursor states. Keep best adapter plus two verified recent resumable bundles; persist from local `/kaggle/working` files rather than treating runtime storage as durable.
4. Overfit 8-16 training examples first. Require finite loss, a downward trend, and substantially improved reproduction of those targets. Failure indicates masking, template, optimization or precision bugs; do not waste a full run.
5. Run the full training split and evaluate validation generation after each epoch. Select checkpoint by validation label macro-F1, then evidence F1 as tie-breaker; log validation loss too. Save each candidate and the selected-checkpoint manifest. Use fixed generation settings for selection and baseline comparison.
6. Test at least one meaningful competing adaptation setting if time allows, e.g. rank 8 versus rank 16 with the same training budget, or 1e-4 versus 2e-4. This supplies observed design evidence. The compulsory base-versus-adapted comparison is separate. Do not launch an open-ended search.
7. Reload the selected adapter in a fresh process, run validation inference, and compare to the base model. Report regressions honestly; a new checkpoint is not automatically better. Do not choose using test performance or replace the required trained system with the base model without explanation.
8. Measure seconds per optimizer step during the pilot and estimate remaining time from batches/accumulation plus validation and snapshot costs. Prove interruption/resume before the full run. Export the selected adapter, configs, environment report and complete logs to persistent storage and the laptop with SHA-256 checksums. Keep the matching base revision/tokenizer locally, reload the exported adapter, and run development inference before accepting training completion.

Outputs: executable training script/config; verified persistent resumable checkpoints; downloaded selected adapter; training history/curve; validation comparison; selected-model manifest with source/runtime/base/tokenizer revisions, precision, hashes and selection reason; local export/reload evidence.

Verify: `python scripts/train.py --config configs/train.yaml --smoke`; then the documented full command; `python scripts/evaluate.py --split validation --model adapted`. Fresh loading reproduces outputs within documented numerical limits; no nontraining patient enters the optimizer.

Done when: a usable adapter exists, validation selection is documented, training logs are complete and the unchanged-model comparison is preserved. No promised accuracy threshold exists in the assignment; provide correct evidence rather than fabricated improvement.

Phase 5 - Make evidence and explanations traceable

Goal: provide defensible criterion results with visible uncertainty. **Inputs:** selected adapter, sentence maps, schemas, parser and validation predictions. **Rubric:** C4, G2 groundwork.

Execution: implement/test the service locally using the exported adapter; reuse it on Kaggle for larger development runs. The service must import without cloud mounts, notebook globals or `/kaggle/working` paths. Use local validation fixtures to verify export parity before final test evaluation.

1. Implement a screening service callable without a UI. It accepts patient text, typed criterion and inference mode, builds the same prompt and returns the stable inference result. Retain source IDs and raw generation.
2. Resolve each predicted ID to exact original patient text. Preserve punctuation and negation. Never ask a model to reconstruct the quote. Mark missing, duplicated and invalid IDs; duplicates may be deduplicated for set metrics but their raw occurrence remains auditable.
3. Add conservative explanation templates. Example: predicted category, cited excerpts, and a short statement that the prediction is based on those sentences. For unknown: evidence is insufficient to establish the criterion. For not-applicable: identify that predicted category and relevant cited context; do not claim a clinical rationale absent from the note. For an empty citation set, explicitly state that no source sentence was cited and that the rationale cannot be verified from citations. A cited sentence is traceability, not proof of clinical entailment.
4. Maintain separate flags for prediction category, parse failure, evidence validity, truncation and missing information. Unknown/NA are legitimate expert categories; runtime failure is an operational state. Keep raw model label for scoring even if the UI flags an inconsistency.
5. Define preliminary patient-trial aggregation in `configs/aggregation.yaml`: any inclusion `not included` or exclusion `excluded` -> preliminary criterion failure; otherwise any unknown, invalid result, unresolved contradiction or truncation -> review needed; otherwise all required inclusion criteria `included` or NA and all exclusion criteria `not excluded` or NA -> preliminary match. Empty criteria -> no assessment. Explain how NA is handled. Do not score this as a ground-truth trial outcome.
6. Build fixtures for age, negation, family history versus patient history, temporal conditions, missing laboratory information, contradictory sentences, exclusion inversion and unsupported citations. Make disagreement visible rather than inventing a tie-breaking medical rule.
7. Begin an error ledger with at least eight distinct development error cases if available. Record annotation ID, gold/prediction, cited/gold evidence, likely cause and consequence. Include unsupported or missing-information cases. Synthetic stress tests can supplement but must be labeled synthetic and do not count as real observed errors. Final-test errors will be analyzed only after freeze.

Outputs: `screening.py`, deterministic evidence lookup, explanation/aggregation policy, integration fixtures, `results/errors/development.md`.

Verify: a synthetic unseen note can be screened from Python; output excerpts exactly match the note; invalid IDs cannot create invented excerpts; an unknown exclusion criterion does not become a positive match. No test cases are used for refinement.

Done when: every displayed result is traceable and operational errors cannot masquerade as clinical categories.

Phase 6 - Implement explicit KV-cached greedy generation

Goal: reuse keys and values correctly, and prove equivalence to the uncached decoder. **Inputs:** selected target/checkpoint, uncached reference, fixed prompts and generation settings. **Rubric:** D1-D3.

Execution: develop portable code and CPU/toy tests locally; run the primary GPU correctness/benchmark grid in Kaggle. Run a small local target-model equivalence set as a delivery gate. Save each environment separately and persist raw token/timing logs after each grid group.

1. Create a cache adapter for the installed Transformers cache class: construct, read sequence length, update through forward output, and crop where supported. Inspect actual tensor shapes and class API. Unit-test behavior before speculative decoding needs rollback [S8].
2. Implement a batch-one cached loop. Prefill the whole prompt once, obtain cache and last-position logits, choose the first token, then feed only newly committed tokens with the returned cache. The prompt length is the first generation position; each new processed token advances it once. Do not feed the whole prompt alongside an already populated cache.
3. Maintain attention masks covering past+new keys. Align position IDs/cache positions with actual token positions, including RoPE offsets. Begin without padding; if adding padding later, compute positions from valid-token masks and test separately. Document why the first sampled token is not yet in the cache until its next forward pass.
4. Build both decoders around the same checkpoint, precision, attention backend, prompt IDs, greedy argmax, EOS and output cap. Disable sampling/dropout and avoid hidden logits processors. A library `generate()` call may act as a third oracle but is not the implementation.
5. Test prompt lengths 128, 512 and 1,024 crossed with output caps 16, 64 and 128. Include real development prompts, synthetic tokens, early EOS and max-length termination. Save output IDs, cache lengths and first differing token/logit margin. Use fixed-step synthetic generation for length-controlled microbenchmarks and normal EOS for application prompts; label these separately.
6. Investigate numerical mismatches using FP32/smaller controlled fixtures, identical attention backend and top-logit margins. Floating-point kernels may differ between full-prefix and incremental calls. Never silently dismiss mismatches or claim token agreement from similar text. Report exact match count over tests.
7. Add timers for prefill and decode, synchronize CUDA around timing, warm up once or more until stable, and repeat at least five measured runs. Report actual generated tokens, end-to-end latency, decode tokens/s and memory. State whether the first token belongs to prefill/TTFT and exclude it consistently from decode throughput.

Outputs: explicit `uncached.py`, `cached.py`, `cache_ops.py`, equivalence tests, `results/cache/correctness.jsonl`, preliminary benchmark CSV and cache diagram/notes.

Verify: `python -m pytest tests/test_cached_decode.py`; `python scripts/benchmark_decoding.py --modes uncached cached --split validation --quick`. Cache length increases exactly by processed tokens; generated tokens agree in the supported deterministic test envelope.

Done when: implementation and saved evidence satisfy all three D items. If a precision/backend produces mismatches, resolve or clearly delimit it before final benchmarking.

Phase 7 - Implement tiled attention with online softmax

Goal: demonstrate mathematically correct attention without materializing the full score matrix in the tiled path. **Inputs:** PyTorch environment and synthetic tensor fixtures; no trained model integration is required. **Rubric:** E1-E3.

Execution: local CPU tests first, then Kaggle GPU measurements on the actual supported backend. Keep local/Kaggle memory and speed tables separate. Do not introduce a CUDA-only package solely because Linux makes installation possible; the portable PyTorch implementation remains sufficient.

1. Implement dense scaled dot-product attention from Q, K, V with logits `Q @ K.T / sqrt(d)`, correct mask, max-subtracted softmax, and multiplication by V. Define shapes `[batch, heads, query_length, head_dim]` and `[batch, heads, key_length, head_dim]`. For GQA testing, explicitly map KV heads to Q-head groups.
2. Implement query/key tiling. For each query block, keep row-wise running maximum m, normalizer l and unnormalized weighted-value accumulator A. Initialize m=-infinity, l=0, A=0. For each key/value tile use:

```
S = Q_tile @ K_tile.T / sqrt(d), with mask applied
m_new = max(m, row_max(S))
alpha = exp(m - m_new)
P = exp(S - m_new)
l_new = alpha * l + row_sum(P)
A_new = alpha * A + P @ V_tile
m, l, A = m_new, l_new, A_new
output = A / l
```

3. Handle all-masked tiles explicitly so `-infinity - -infinity` cannot create NaNs. Preserve state for such rows; define a fully masked output row as zero and test it in both implementations. Use FP32 accumulation for running statistics/weighted sums in low precision.
4. Apply causal masking by **absolute** query/key positions: a key is visible only when its position is no later than the query's position. For cached-style rectangular tests, queries start at the past-prefix offset. A simple square `tril` or default backend causal flag may have different rectangular alignment; supply an explicit mask when needed.
5. Test lengths 1, 17, 64, 129 and 256, head dims 32/64/128, several heads and tiles 16/32/64. Include nondivisible tiles, causal/noncausal, large logits, fully masked rows and rectangular cached-offset cases. Compare maximum and mean absolute error and relative error away from zero.
6. Starting tolerances: FP32 `atol=1e-5, rtol=1e-4`; FP16/BF16 `atol=1e-2, rtol=1e-2`. Justify using observed error scales; do not loosen tolerances to conceal a masking bug. Use a high-precision small oracle when necessary.
7. Benchmark 128/512/1,024 lengths and, if safe, 2,048; warm up, synchronize, repeat, reset memory peaks and isolate each implementation. Show that tiled code does not allocate an entire N x N score tensor. Compare PyTorch SDPA optionally, recording actual backend [S12]. An educational Python-loop implementation may be slower because it lacks fused-kernel scheduling and data movement optimizations [S9-S10].

Outputs: `dense.py`, `tiled.py`, numerical tests/results, speed/memory CSV, one error plot and one runtime/memory plot. Record whether this is standalone or integrated. Do not label Python code as the production FlashAttention kernel.

Verify: `python -m pytest tests/test_attention.py`; `python scripts/benchmark_attention.py --config configs/attention_bench.yaml`. No NaNs, future-token leakage or full score-matrix allocation in the tiled algorithm.

Done when: both implementations pass multiple shapes/masks and measured comparisons are reproducible, regardless of which is faster.

Phase 8 - Implement greedy speculative decoding

Goal: use a smaller draft while reproducing the target's ordinary greedy token sequence. **Inputs:** selected fine-tuned target, verified smaller draft, tokenizer compatibility evidence, cached target reference and tested cache operations. **Rubric:** F1-F3.

Execution: develop rejection/state tests locally; run the primary k/length sweep on the Kaggle GPU and persist results. Then prove real target/draft equivalence and peak-memory fit in a small local demo envelope. Never compare speculative cloud timing with ordinary laptop timing as a speedup.

This is deterministic matching of argmax tokens. It does not require probabilistic acceptance ratios or sampling corrections from the stochastic versions of the papers [S13-S14]. The target is the selected adapted model, not an unadapted proxy.

Commit a precise state invariant

At the start of each iteration, both caches represent exactly the committed prefix x. Each model also has its next-token logits after x. Caches may have different internal shapes but must represent identical token IDs and length. Cache mutations are explicit; avoid accidental aliasing when saving state.

1. Prefill both models on x. Select draft block size k=4 initially, bounded by the remaining output budget. Draft d1..dk greedily, processing each through the draft so its cache contains x+d1..dk and its next logits are valid.
2. Verify the block in **one target forward pass** on d1..dk with the target prefix cache. The saved target logits after x predict d1. Verification logits after d_i predict d_(i+1), and logits after d_k predict an optional bonus token. This alignment prevents the common one-token shift bug.
3. Compare draft tokens with target argmax tokens until the first mismatch. If r draft tokens match and d_(r+1) fails, commit only d1..dr and the target correction token c from that exact position. Discard every later candidate.
4. On rejection, crop both caches to `len(x)+r`, then process c through both models to restore the invariant and next-token logits. Cropping to include the rejected token is wrong. Update positional counters and masks with the crop.
5. If all k match, retain their caches. If budget permits, choose a bonus token from target logits after d_k, commit it and process it through both models. If no room remains, stop without the bonus. Alternatively omit bonus consistently and document the algorithm; never exceed the requested output cap.
6. Stop immediately when a committed token is an EOS; discard candidates after it. Never accept a later token once EOS is committed. A correction token may itself be EOS. Respect the same full stop-ID set as ordinary greedy.
7. Implement cache crop for the actual library class and unit-test key/value lengths and positional state. A recompute-from-prefix fallback can establish correctness while debugging, but final measurements must disclose it; avoid reporting a cache-reuse speedup when rebuilding caches.

Phase 8 - Validation and measurements

Tests and measurements

Use forced/mock logits to cover zero accepted tokens, partial acceptance, full acceptance, repeated rejection, correction EOS, draft EOS, bonus EOS, and one-token remaining budget. Then use the real pair for k=1/2/4/8 across at least three prompt/output lengths and actual development clinical prompts. Verify complete output token equality to the same adapted target's cached greedy path. Save disagreement position, surrounding tokens, logits and cache lengths for failures.

Acceptance rate = accepted draft tokens / all proposed draft tokens. Also log proposed/accepted counts, acceptance per block position, target verification calls, correction/bonus tokens, prefix lengths and actual output tokens. End-to-end timing includes both prefills and all draft, verify, crop/synchronization and correction costs; report warm-model load-excluded latency separately from cold application startup. Speedup = target cached-greedy latency / speculative latency on identical prompts and output lengths.

Outputs: `speculative.py`, adversarial state-machine tests, real-model token equivalence log, acceptance traces, latency/throughput/memory CSV and algorithm diagram.

Verify: `python -m pytest tests/test_speculative.py`; `python scripts/benchmark_decoding.py --modes cached speculative --draft-k 1 2 4 8 --split validation`. The draft is smaller, target matches the selected checkpoint, and measured peak memory includes both models.

Done when: drafting, block verification, rejection correction and cache synchronization are implemented and defended. A slowdown can earn full marks if correctness and measurements are convincing; do not manufacture a speedup.

Phase 9 - Freeze settings, run final evaluation and consolidate evidence

Goal: produce a trustworthy final scientific comparison. **Inputs:** selected checkpoint, all passing correctness tests, immutable split and validation-selected settings. **Rubric:** B4, C1-C4, D3, E3, F3.

Execution: use one recorded primary research environment, normally Kaggle, for the paired final quality evaluation and within-device benchmark groups. Local-app smoke/correctness checks use validation/synthetic fixtures and must already pass before opening test outputs. Download the frozen manifest and predictions after each completed job.

1. Freeze Git commit, dataset revision, split hash, model/tokenizer revisions, adapter hash, prompt/template, truncation, parser/aggregation policies, precision, backend, decoding caps, metric conventions and the primary research/runtime profile in `results/frozen_manifest.json`. Include validated local deployment identity and any explicitly separate variant. No quality-related decision uses test outcomes.
2. Use a fixed validation/synthetic benchmark suite for performance sweeps. Cache modes: 128/512/1,024 prompts x 16/64/128 outputs. Speculation: k=1/2/4/8, same prompts and target. Attention: multiple safe shapes/tiles. Add real clinical prompt timings with normal EOS. Report unavailable/OOM cells honestly.
3. Warm up and use at least five measured runs, randomizing mode order where practical. Synchronize GPU work, reset request/cache state and peak counters, and report median plus IQR/range. Label each group with actual GPU, dtype, backend and session ID. After a GPU-changing reconnect rerun every compared mode for that group; do not average unlike accelerators or include artifact/checkpoint transfers inside inference timers.
4. Record `prefill_ms`, `decode_ms`, TTFT, total request latency, actual output count, tokens/s, peak allocated/reserved VRAM and CPU RAM where available. Distinguish loaded-weight baseline from incremental memory. For speculation include draft+target residency and both prefills. State whether tokenization, JSON validation and UI/export are included; keep model-only and end-to-end measures separate.
5. Run unchanged/adapted targets on the **same 206 held-out rows**, with identical prompt, preprocessing, precision, greedy mode and caps in the primary environment. Persist per-row outputs before inspecting gold comparisons. Resume interrupted evaluation using annotation ID plus full frozen-config/model hashes, never missing rows or mismatching variants. Produce all required quality/validity/truncation results with no row filtering. A later exact rerun is reproducibility, not a new tuning opportunity.
6. Analyze at least eight distinct real errors across validation/test, clearly marking split. Include unknown/missing-information, unsupported evidence, exclusion polarity, temporal/negation or contradiction where present. Do not claim categories observed if they did not occur. If fewer than eight test errors exist, use validation errors to reach eight; if fewer than eight genuine errors exist overall, document that rather than inventing them and add separately labeled stress failures.
7. Do not tune after inspecting test results. A confirmed implementation bug may justify a versioned rerun, but preserve the original outputs, explain the fix and acknowledge prior test exposure. Any additional test-derived redesign becomes exploratory and must not be presented as untouched held-out confirmation.
8. Generate plots/tables from machine-readable logs: training curves; base/adapted metric table; per-type confusion matrices; evidence metrics; cache latency/memory; attention error/time/memory; speculative acceptance/speedup; validity/truncation summary. Preserve CSV/JSON sources for every figure.

Outputs: frozen manifest, complete base/adapted test predictions, evaluation tables, eight-case error ledger, all repeated benchmark logs, figures and a short findings note.

Verify: `python scripts/evaluate.py --split test --frozen-manifest results/frozen_manifest.json --allow-final-test`; `python scripts/summarize_results.py`. Recompute tables solely from saved predictions/logs and verify every expected annotation is present. An independent rerun of the same frozen configuration is permitted for reproducibility, not selection.

Done when: scientific results can be traced to a checkpoint, config, commit and raw log; all three inference paths have correctness evidence and honest measurements.

Phase 10 - Deliver the standalone browser application

Goal: let an evaluator screen unseen cases and compare inference without executing code manually. **Inputs:** stable screening service, trained checkpoint, implemented decoders, exported benchmark results and frozen metadata. **Rubric:** G1-G4.

Execution: host entirely on the laptop's localhost using exported artifacts. No active Kaggle session, notebook execution, cloud authorization, share tunnel or cloud-GPU allocation may be required for ordinary use. The app shows the local environment identity; historical cloud charts carry their own hardware labels. Free Kaggle is reserved for research jobs [S15].

1. Build a lightweight Gradio interface backed by the existing service. No new training or model logic belongs in UI callbacks. Load the target once, use a request queue, serialize GPU jobs and support explicit draft loading for speculative mode.
2. Provide patient entry, visible numbered sentences, criterion type and criterion text, and a small editable multi-criterion table. Allow selection of a bundled example trial whose criteria are saved with provenance or entry of custom criteria. A live trial-search API is optional and must never be necessary for the demo. Do not use test records as polished examples.
3. Show per-criterion category, original criterion, cited IDs, exact excerpts, explanation, uncertainty/invalid-output/truncation warnings and preliminary rule-based overall outcome. Expose that aggregate as a screening rule, not a separately learned or labeled trial verdict.
4. Add uncached/cached/speculative mode selection. Run the same current prompt/settings through the actual local modes to show token equality, latency, throughput, memory and draft acceptance. Use validated batch-one limits and no duplicate target loads. Add small live dense/tiled attention examples. Historical Kaggle charts are supplementary and labeled by GPU/session; they cannot stand in for live app comparison or be combined with local times to calculate speedup.
5. Validate blank inputs, missing types, oversized notes/criteria, conflicting sentence numbering, output-cap exhaustion, unavailable weights, OOM and model errors. Return actionable messages and recover for the next request. Do not silently send a different model/precision as a fallback. Treat instructions embedded in patient text as data.
6. Export a readable self-contained HTML screening report, plus JSON if convenient. Include entered text or an explicitly chosen privacy-preserving representation, numbered evidence, criteria, labels, explanations, flags, aggregate rule, timestamp/timezone, model/adapter/prompt revisions, decoding settings and timings. HTML can be printed to PDF; a direct PDF export is optional because the assignment only requires a readable report.
7. Display the exact notice: **Educational screening prototype; not a clinical enrollment decision.** Do not persist uploaded notes by default or commit them to Git. Use bundled synthetic/public examples for the demo.
8. Create `Start_App.cmd` to call `.venv\Scripts\python.exe app\main.py` and open localhost after the server is ready. Provide a setup/download launcher for initial provisioning and clear startup documentation. Setup can be documented, but ordinary assessment after setup must not require typing commands in a terminal/IDE/notebook. Handle spaces in paths and show useful startup errors. No paid API key or hosted GPU must be needed.

Outputs: app source, launchers, setup/startup guide, bundled development/synthetic examples, exported sample report and screenshots.

Verify: disconnect Kaggle, then double-click the local launcher from a fresh login/shell. Screen an unseen synthetic note, enter/select criteria, compare modes, inspect evidence and export. Test invalid/long input followed by a valid request. Disable networking after downloading weights and repeat. Record actual laptop startup/response times; no notebook or Google login is needed.

Done when: an evaluator can use all core functions in a browser and recover from invalid inputs without editing code. If integration reveals a model-affecting bug after test freeze, use the versioned correction policy from phase 9.

Phase 11 - Write the paper, package the submission and rehearse

Goal: make every claim reviewable and every deliverable accessible. **Inputs:** completed artifacts, measured results, app and source ledger. **Rubric:** H1-H3 and final A3 evidence.

Execution: write/package locally using downloaded, verified results. Record the demo on the laptop with no active Kaggle runtime. Include the thin research notebook, both environment profiles, interruption/resume procedure and artifact transfer manifest in reproducibility documentation.

1. Prepare an IEEE-format paper in LaTeX using an appropriate IEEE template. Include title/author, abstract/keywords, introduction/problem, related work, materials/methods, experimental setup/metrics, results/discussion, limitations, conclusion, references and a brief AI-use appendix. This master-plan PDF is not the assignment's required research-paper PDF.
2. Methods must cover exact dataset/split, leakage controls, sentence numbering, model architecture diagram, fine-tuning objective/settings, prompt/output schemas, truncation, attention recurrence, cache flow, speculative algorithm and application workflow. State whether custom attention runs inside the target or independently.
3. Include measured training curves, metric/benchmark tables, numerical error and memory graphs, eight real error analyses and app screenshots. Refer to and interpret every figure/table. Explain slower speculation or tiled attention honestly. Do not paste screenshots of results instead of scientific tables.
4. Discuss small data, eight test patients, imbalance, pretrained-data contamination, trial overlap, missing evidence and clinical limitations. Identify actual Kaggle GPU/RAM and local GPU/RAM separately, note interruptions and measured transfer/setup costs, and explain device-specific timing. Do not claim a generic Kaggle speedup from unlike hardware comparisons. Acknowledge that source-linked excerpts alone do not establish medical correctness.
5. Publish the GitHub repository when ready, with executable scripts, dependency lock, configs, tests, manifest, logs, app, reproducibility steps and licenses/attribution. Host adapters/large weights elsewhere with a working download script, exact base revision, checksum and license notice. Verify a fresh download; do not commit tokens or large caches.
6. Share the LaTeX source project link (GitHub `report/` may serve this if accessible), compiled IEEE report PDF, repository URL, model/adapter link, split manifest, benchmark logs and **5-7 minute video link**. Verify all links with the evaluator's intended access, not only the owner's session.
7. Suggested six-minute video: 0:00-0:40 problem/resources; 0:40-2:30 unseen patient and criteria with evidence/unknown case; 2:30-3:50 live decoding comparison and attention view; 3:50-4:30 export and invalid-input recovery; 4:30-5:30 before/after results and limitation; 5:30-6:00 repository/reproducibility. Use actual runs, not simulated charts.
8. Rehearse live: explain inclusion/exclusion labels; trace one evidence ID; derive online-softmax state; show cache lengths/position masks; walk a rejected speculative token; discuss why a speedup failed; reload adapter; execute an unseen case. Be ready to change a small component and rerun a focused correctness test.
9. Finish `SUBMISSION.md` with artifact URLs, startup steps, environment/model revisions, reproducibility commands and known limitations. Upload through Classroom before the verified deadline, confirm attachment/link access and save the submission receipt. Confirm live-demo attendance.

Outputs: `report/main.tex`, bibliography/figures, compiled research paper, accessible source project, complete repository release, model artifact, demonstration video and `SUBMISSION.md`.

Verify: clean-environment setup + adapter download + app launch; report compiles without missing references; plots match logs; links resolve; video length is 5-7 minutes; no required file exists only on the author's machine.

Done when: the checklist in section 5 is satisfied, the submission receipt is saved, and the live-demo commitment is scheduled. Full credit remains an instructor judgment; this plan covers the stated observable standards.

4. Risk controls and scope discipline

Risk / trigger	Required response
OOM during target training	Record peak and failing shape; checkpoint; apply resource ladder. Preserve train split, objective and comparison identity.
Target+draft do not fit	Lower safe input/output envelope, validate memory, or use a free GPU. Any precision/model change needs a matching ordinary-target reference.
Free GPU unavailable	Time-box setup to 20-30 minutes, continue CPU/local fallback work, and preserve the same configs/split. No paid environment dependency.
Kaggle disconnects / runtime storage disappears	Restore the newest checksum-verified persistent bundle; verify optimizer and data position. Keep the selected adapter and all logs locally.
Reconnected GPU differs	New environment/session ID; rerun all compared benchmark modes in that group. Never merge unlike-device times into a speedup.
Cloud model fits but local demo fails	Resolve before test freeze using the phase-1 local gate. Keep the small pair; validate any deployment variant explicitly.
Kaggle becomes an app-hosting dependency	Move hosting to local localhost and verify offline startup. Keep notebook orchestration confined to research.
Training loss does not fall on tiny subset	Inspect answer loss mask, template, LR, precision and trainable parameters before full training.
Accuracy looks high but macro-F1 is weak	Inspect per-class counts and confusion matrices; do not collapse unknown/NA or drop rare labels.
Excess malformed output	Diagnose with validation only; adjust fixed output cap/schema/prompt before final freeze. Keep all failures in metrics.
Cache/speculative token mismatch	Stop performance claims; inspect first divergence, logits, cache length, positions and rollback.
Tiled attention slower than dense	Report it; test memory benefit and correctness. Kernel fusion is outside compulsory scope.
Speculation slower than cached greedy	Report acceptance/cost decomposition. Correctness earns implementation credit without a speedup.
Test accidentally exposed during tuning	Preserve provenance, disclose exposure and stop describing subsequent tuning as independent held-out confirmation.
Deadline pressure	Cut optional deployment, live trial API, extra models, elaborate CSS and large hyperparameter sweeps; retain every mandatory task and release evidence.

Avoid unnecessary scope: no vector database, RAG corpus, paid API, 7B+ model, Kubernetes, production authentication or custom CUDA build is required. Public trial metadata is optional application support, never a replacement supervised benchmark. Prototype behavior must remain clear about missing information and educational use.

5. Full-credit deliverables and rubric traceability

This section is the authoritative completion checklist for this plan. A standalone copy is provided in `deliverables_checklist.md`. Each subcriterion is worth 5 marks in the detailed rubric, totaling 135. Filenames below are proposed paths to create; check them off only when the actual evidence exists.

Code	Required evidence	Completion artifact
A1	At least five credible sources; critical comparison and limitations connected to design	`research/sources.md`, paper related work and bibliography
A2	Two candidates; accurate architecture/license; measured cloud-training and local-inference feasibility	model comparison, `results/environments/`, cloud/local fit logs
A3	Alternatives, rejected options, observed failures and evidence-based choices	`research/decision_log.md`, experiment comparisons, paper discussion
B1	Prescribed data; exact 37/8/8 seed-42 patient split; IDs/counts and leakage checks	`data/splits/manifest.json`, data hashes, tests
B2	Patient/type/criterion inputs; expert labels and sentence-ID targets; no GPT leakage	preparation script, schemas, prompt audit, preprocessing notes
B3	Executable training, reproducible config, verified resume/export, adapters and training logs/curves	script/config, Kaggle runner, resumable bundles, manifest, downloaded adapter
B4	Same unchanged/adapted model comparison on identical validation/test examples and decoding settings	paired predictions, metric tables, manifests
C1	Inclusion/exclusion labels including unknown/NA; held-out accuracy/macro-F1	overall and per-type test summary
C2	Per-class precision/recall/F1 and confusion matrices, overall and by type	metric JSON/CSV, confusion-matrix figures
C3	Original sentence-ID retrieval; micro/macro evidence metrics and empty-set rules	evidence scorer/tests, saved predictions, metric tables
C4	Exact evidence-linked explanations; at least eight distinct genuine errors with missing/unsupported cases	screening output, error ledger, paper analysis
D1	Explicit forward/cache reuse/update, masks, positions and cache growth	`decoding/cached.py`, cache notes/diagram
D2	Greedy token equivalence across at least three input/output lengths; mismatch investigation	test suite and token-level correctness logs
D3	Warmed/synchronized repeated timing and peak memory; same-device mode comparisons	raw cache CSV/JSON, GPU/session metadata and plots
E1	Correct stable scaled dot-product reference with scaling/masking	`attention/dense.py`, numerical tests
E2	Actual blocked attention and online maximum/normalizer/value accumulation	`attention/tiled.py`, recurrence in report
E3	Multiple shapes/causal masks; max/mean error; honest speed/memory comparison	attention tests, raw logs and figures
F1	Smaller compatible draft/target; tokenizer check, settings and memory justification	model manifest, compatibility test, feasibility record

Code	Required evidence	Completion artifact
F2	Own drafting, block verification, accepted-prefix/rejection correction and cache synchronization	`decoding/speculative.py`, state-machine tests
F3	Target-greedy equality, acceptance/latency/memory, within-device speedup or slowdown	cloud benchmark traces, local correctness/fit log, paper analysis
G1	Standalone local browser UI, startup/model loading and recovery; independent of Kaggle	app source, launcher, offline/disconnected-Kaggle verification
G2	Unseen note + selected/entered criteria; decisions, exact evidence, uncertainty and preliminary aggregate	working app, integration cases, video
G3	Live meaningful inference comparison and experiment visibility	compare action, attention view, displayed token agreement/timing
G4	Readable UI and downloadable screening report with provenance; invalid-input recovery	exported HTML report, screenshots, recovery evidence
H1	Complete IEEE LaTeX research paper; interpreted methods/results/plots, limitations and citations	compiled paper PDF, LaTeX source project link
H2	GitHub source, both runtime profiles, research launcher/resume guide, model/split/logs and video	repo/model URLs, environment locks, transfer hashes, guide, 5-7 minute video
H3	Individual understanding; explain, execute or modify components and handle unseen cases live	defense notes, functioning local app, live-demo attendance

Final submission bundle

- [ ] IEEE research-paper PDF with AI-use appendix and linked accessible LaTeX source project.
- [ ] GitHub URL containing complete source, environment lock, data preparation, training/evaluation scripts, all three decoders, both attention implementations, app and correctness tests.
- [ ] Trained adapter/weights link, exact base-model/tokenizer revisions, license notices, checksums and tested download/loading method.
- [ ] Published split manifest with patient IDs, annotation membership/counts, seed/procedure and leakage checks.
- [ ] Paired baseline/adapted validation and held-out predictions, complete classification/evidence results, training logs/curves and error analysis.
- [ ] Raw benchmark logs, numerical/token correctness evidence, machine-readable summary tables and interpreted figures.
- [ ] Clear setup/reproduction guide and double-click browser-app launcher; sample downloadable screening report with provenance and educational notice.
- [ ] Thin Kaggle research notebook, tested local/cloud environment profiles, resumable-checkpoint instructions, durable artifact/export hashes and a successful local adapter reload. These support the rubric's reproducibility requirements; they are workflow additions, not extra instructor-assigned marks.
- [ ] Actual GPU/session labels on research benchmarks, within-device speedup calculations, and independent local app/live-comparison checks with Kaggle disconnected.
- [ ] Accessible 5-7 minute recorded demonstration link, correct sharing permissions and confirmed live-demo attendance.
- [ ] `SUBMISSION.md` linking every artifact, known limitations and the exact release commit; Classroom submission receipt.

6. Verified research starting points

Model/dataset/research sources were checked on 8 November; Kaggle and environment guidance was checked on 9 November 2026. Pin revisions and installed versions; live documentation may change. These support the plan, not measured results. Read relevant methods/code and add BibTeX entries.

- **[S1] Dataset:** NCBI TrialGPT Criterion Annotations. Required supervised benchmark; source card identifies fields and the public-domain designation. The current card reports 1,015 examples. Exact split counts in this plan were independently computed using the assignment procedure.
- **[S2] Clinical reasoning:** Jin et al., Matching patients to clinical trials with large language models, Nature Communications (2024), and official reference repository. Use for criterion reasoning/evidence and limitations. Its broader study is not your fixed-split experiment, and its results cannot be copied as your own.
- **[S3] Target specification:** Qwen2.5-1.5B-Instruct model card and config. Architecture, context, license and parameter source for the proposed target.
- **[S4] Draft/candidate specification:** Qwen2.5-0.5B-Instruct model card and config. Smaller candidate and draft; compatibility still requires a local test.
- **[S5] Architecture:** Qwen2 Technical Report. Background on this decoder family; use the individual Qwen2.5 configs for checkpoint-specific details.
- **[S6] Adaptation:** Hu et al., LoRA and PEFT LoRA documentation. Motivation and implementation support for low-rank adaptation; validate behavior against the installed package.
- **[S7] Quantization fallback:** Dettmers et al., QLoRA. Frozen quantized base plus trainable adapters. Quantization changes numerical behavior and adds software constraints; measure rather than assume feasibility.
- **[S8] Cache implementation:** Transformers: How caching works. Cache lifecycle, masks and positions; inspect the version you install.
- **[S9] Attention:** Dao et al., FlashAttention. IO-aware tiling motivation. A Python online-softmax exercise is not the fused production kernel.
- **[S10] Stable memory-efficient attention:** Rabe and Staats, Self-attention Does Not Need O(n^2) Memory. Background for blockwise stable accumulation and memory reasoning.
- **[S11] Environment:** PyTorch official installation guide. Select a wheel compatible with the actual OS/Python/GPU driver; validate in the project environment.
- **[S12] Optional backend reference:** PyTorch scaled_dot_product_attention documentation. Compare supported optimized backends separately from the custom implementation, including mask conventions.
- **[S13] Speculation:** Leviathan et al., Fast Inference from Transformers via Speculative Decoding. Core draft/verification motivation; distinguish the required greedy specialization from probabilistic sampling.
- **[S14] Independent speculation reference:** Chen et al., Accelerating Large Language Model Decoding with Speculative Sampling. Additional verification/performance perspective; published speedups on different hardware are not your results.
- **[S15] Compute service:** Official Kaggle CLI and kernel metadata, checked 9 November 2026. Quota, internet, hardware and persistence remain to be verified.
- **[S16] Runtime compatibility:** Transformers installation and offline-cache guidance and PEFT quantized-adaptation guidance, checked 9 November 2026. Use installed-version-compatible code, explicit caches and an offline local load test.

7. Handoff prompt for completing one phase

Copy the selected phase in full and provide only its listed inputs plus the completed handoff files. The agent does not need later phases to finish the selected task.

```
Implement Phase [number] using its stated input artifacts and contracts.
Read the current repository instructions and completed handoff for each input.
Preserve the fixed patient split, source sentence IDs and leakage controls.
Use the phase's declared local/Kaggle route and portable runtime profile.
Persist cloud outputs with verified hashes; no future phase may require
an active notebook session. Keep speed comparisons within one device.
Do not inspect test content unless this is the frozen final-evaluation phase.
Implement the listed outputs and run the phase's acceptance checks.
Do not substitute future work, mock results or library switches for required code.
Record exact commands, versions, config/artifact hashes, failures and evidence
in handoffs/phase_XX.md. Stop only after the acceptance gate passes or clearly
identify a real unresolved blocker. Do not claim unrun checks passed.
```

The output names and schemas in each phase are its handoff boundary. If an implementation must change a contract, document and version that change in the current phase, update its consumers when they are implemented, and preserve old results where comparisons would otherwise become ambiguous.